

Homework 2 — Stochastic Gradient Descent

Statistical and Mathematical Methods for AI

Indice

1	Obiettivo dell'homework	2
2	Esercizio 1: SGD vs GD su un problema di regressione 1D	3
3	Esercizio 2: Varianza del gradiente stocastico (esperimento 1D)	8
4	Esercizio 3: SGD in 2D con rumore	12
5	Esercizio 4: Un progetto ML con SGD	21

1 Obiettivo dell'homework

Questo homework estende la Gradient Descent (HW1) al caso in cui il gradiente esatto non è disponibile o è troppo costoso da calcolare ad ogni passo, introducendo la **Stochastic Gradient Descent** (SGD) e il suo compromesso fondamentale tra rumore e costo computazionale:

- **Esercizio 1** confronta GD a batch pieno e SGD a mini-batch su un problema di regressione lineare, mostrando come la dimensione del batch influenzi rumore, numero di aggiornamenti per epoca e accuratezza finale.
- **Esercizio 2** misura *quantitativamente* la varianza del gradiente stocastico al variare della dimensione del batch, verificando empiricamente la legge $\text{Var}(g) \propto 1/N_{\text{batch}}$.
- **Esercizio 3** studia l'effetto del rumore su una funzione non convessa a due valli, mostrando come un livello di rumore moderato possa aiutare a sfuggire da regioni piatte, mentre un rumore eccessivo impedisce la convergenza fine.
- **Esercizio 4** applica GD e SGD a un problema di regressione reale (costi assicurativi), confrontando le due famiglie di metodi su dati non sintetici.

Il filo conduttore è che il gradiente di mini-batch g_k è uno stimatore **non distorto** ma **rumoroso** del gradiente vero $\nabla \mathcal{L}$: la sua varianza scala come $1/N_{\text{batch}}$, e tutto il comportamento qualitativo della SGD (curve rumorose, capacità di sfuggire a regioni piatte, minor costo per aggiornamento) discende da questo singolo fatto statistico.

2 Esercizio 1: SGD vs GD su un problema di regressione 1D

Consegna

Consider the synthetic dataset

$$x^{(i)} = \frac{i}{N}, \quad y^{(i)} = 2x^{(i)} + 1 + \varepsilon^{(i)}, \quad \varepsilon^{(i)} \sim \mathcal{N}(0, 0.01),$$

with $N = 200$. We model the data with a linear function $f_{\Theta}(x) = \Theta_0 + \Theta_1 x = \Theta^T \tilde{x}$, con $\tilde{x} = [1, x]$.

1. Implement the MSE loss $\mathcal{L}(\Theta) = \frac{1}{N} \sum_{i=1}^N (f_{\Theta}(x^{(i)}) - y^{(i)})^2$.
2. Implement **full GD** and **SGD** (mini-batch) using batch sizes $N_{\text{batch}} \in \{1, 10, 50, N\}$ (l'ultimo recupera la GD).
3. Plot per ogni metodo: la curva della loss (loss vs epoca), la traiettoria dei parametri (Θ_0, Θ_1) nel piano dei parametri.
4. Discuss:
 - Why GD is smooth but slow for large N ,
 - Why SGD is noisy but progresses faster,
 - How batch size affects the noise level and convergence stability.

```
In [1]:
1  N = 200
2  x = np.arange(1, N+1) / N
3  noise = rng_global.normal(0, 0.1, size=N)    # std = sqrt(0.01) =
      0.1
4  y = 2*x + 1 + noise
5
6  X_tilde = np.column_stack([np.ones(N), x])    # shape (N, 2)
7
8  def mse_loss(theta, X, y_):
9      pred = X @ theta
10     return np.mean((pred - y_)**2)
11
12 def mse_grad(theta, X, y_):
13     pred = X @ theta
14     return (2.0 / X.shape[0]) * (X.T @ (pred - y_))
15
16 # soluzione esatta ai minimi quadrati (il minimo della loss
    empirica)
17 theta_ls = np.linalg.lstsq(X_tilde, y, rcond=None)[0]
18 print(f"theta ai minimi quadrati = {theta_ls}, L = {mse_loss(
    theta_ls, X_tilde, y):.6f}")
19 print(f"parametro vero (1, 2)      = [1. 2.], L = {mse_loss(np.
    array([1.,2.]), X_tilde, y):.6f}")
20
21 # condizionamento del problema: Hessiana costante H = (2/N) X^T X
22 H1 = 2.0 * X_tilde.T @ X_tilde / N
23 ev1 = np.linalg.eigvalsh(H1)
24 print(f"autovalori dell'Hessiana = {ev1}, kappa = {ev1[-1]/ev1
    [0]:.1f}, "
```

25

```
f"eta massimo per il full GD = {2/ev1[-1]:.3f}")
```

Out[1]:

```
1 theta ai minimi quadrati = [1.01150077 1.98015034], L = 0.009206
2 parametro vero (1, 2) = [1. 2.], L = 0.009241
3 autovalori dell'Hessiana = [0.13120609 2.54046891], kappa = 19.4, eta
  massimo per il full GD = 0.787
```

La loss MSE per un modello lineare è una quadratica in Θ (stessa struttura di HW1), quindi si può calcolare esattamente il suo minimo $\hat{\Theta}$ tramite minimi quadrati e la sua Hessiana $H = \frac{2}{N}X^T X$ (costante, come nel caso quadratico di HW1 §Esercizio 3). Qui $\kappa \approx 19.4$: un condizionamento moderato, non estremo come nell'Esercizio 3 di HW1 ($\kappa = 25$) ma comunque non banale. La soglia di stabilità per il full GD è $\eta < 2/\lambda_{\max} \approx 0.787$: lo step scelto sotto, $\eta = 0.5$, sta ampiamente all'interno di questa soglia.

Da notare: $\hat{\Theta} = (1.012, 1.980)$ è leggermente diverso dal parametro vero $(1, 2)$ — è normale, è l'effetto del rumore campionario $\varepsilon^{(i)}$ nei dati: $\hat{\Theta}$ è il minimo della loss *empirica*, non del modello vero.

In [2]:

```
1 def run_gd_sgd(X, y_, theta0, eta, batch_size, n_epochs, seed=0):
2     """GD/SGD unificati: batch_size == N restituisce il full-batch
3     GD.
4     Registra theta dopo ogni update e la loss sull'intero dataset
5     dopo ogni epoca.
6     """
7     rng = np.random.default_rng(seed)
8     theta = np.array(theta0, dtype=float)
9     thetas = [theta.copy()]
10    epoch_losses = [mse_loss(theta, X, y_)]
11    N_ = X.shape[0]
12    for epoch in range(n_epochs):
13        idx = rng.permutation(N_)
14        for start in range(0, N_, batch_size):
15            b_idx = idx[start:start+batch_size]
16            g = mse_grad(theta, X[b_idx], y_[b_idx])
17            theta = theta - eta * g
18            thetas.append(theta.copy())
19            epoch_losses.append(mse_loss(theta, X, y_))
20    return theta, np.array(thetas), np.array(epoch_losses)
21
22 theta0 = np.array([0.0, 0.0])
23 eta1 = 0.5
24 n_epochs1 = 40
25 batch_sizes1 = [1, 10, 50, N]
26
27 results1 = {}
28 print(f"{'batch':>6} {'update/epoca':>13} {'update totali':>14} {'
29     theta finale':>26} "
30     f"{'loss':>10} {'dist. da theta_ls':>18}")
31 for bs in batch_sizes1:
32     theta_f, thetas, losses = run_gd_sgd(X_tilde, y, theta0, eta1,
33         bs, n_epochs1, seed=1)
34     results1[bs] = {'theta_final': theta_f, 'thetas': thetas, '
35         losses': losses}
```

```

31     print(f"{bs:>6} {int(np.ceil(N/bs)):>13} {len(thetas)-1:>14} "
32           f"{np.array2string(theta_f, precision=4):>26} {losses
33           [-1]:>10.5f} "
           f"{np.linalg.norm(theta_f - theta_ls):>18.4f}")

```

Out [2]:

	batch	update/epoca	update totali	theta finale	loss
1		dist. da theta_ls			
2	1	200	8000	[0.9921 1.9552]	0.01028
		0.0316			
3	10	20	800	[1.0214 1.9816]	0.00932
		0.0100			
4	50	4	160	[1.0061 1.9786]	0.00924
		0.0056			
5	200	1	40	[1.0512 1.9063]	0.00967
		0.0838			

Un risultato non banale, visibile solo grazie all'ultima colonna. La distanza finale da $\hat{\Theta}$ non è monotona in N_{batch} : è massima per batch= 1 (0.0316, il più rumoroso) ma è quasi altrettanto grande per il full GD (0.0838, il meno rumoroso!), mentre il minimo si ottiene con batch intermedi (0.0056 per batch= 50). La spiegazione sta nella colonna "update totali": con lo stesso numero di epoche (40), batch= 1 fa 8000 aggiornamenti (ma ciascuno rumorosissimo), mentre il full GD ne fa solo 40 (esatti ma pochissimi — non ha ancora fatto in tempo a convergere del tutto). I batch intermedi bilanciano meglio le due cose: abbastanza aggiornamenti, abbastanza poco rumore per aggiornamento.

In [3]:

```

1  fig, axes = plt.subplots(1, 3, figsize=(18, 5))
2  colors1 = {1: 'tab:red', 10: 'tab:orange', 50: 'tab:green', N: 'tab
   :blue'}
3
4  ax = axes[0]
5  for bs in batch_sizes1:
6      label = f'batch={bs}' + (' (full GD)' if bs == N else '')
7      ax.semilogy(results1[bs]['losses'], color=colors1[bs], label=
        label)
8  ax.axhline(mse_loss(theta_ls, X_tilde, y), color='black', ls=':',
        lw=1,
9             label=r'$\mathcal{L}(\hat{\Theta})$ (minimo)')
10 ax.set_xlabel('epoca'); ax.set_ylabel(r'$\mathcal{L}(\hat{\Theta})$ (log)
    ')
11 ax.set_title('Loss vs epoca')
12 ax.legend(fontsize=8)
13
14 for ax2, zoom in zip(axes[1:], [False, True]):
15     for bs in batch_sizes1:
16         thetas = results1[bs]['thetas']
17         label = f'batch={bs}' + (' (full GD)' if bs == N else '')
18         ax2.plot(thetas[:,0], thetas[:,1], '--', color=colors1[bs],
19                 alpha=0.7, lw=1, label=label)
19     ax2.scatter([theta_ls[0]], [theta_ls[1]], color='black', marker
20                ='*', s=140, zorder=5,
21                label=r'$\hat{\Theta}$ (minimi quadrati)')
22     ax2.scatter([1], [2], color='gray', marker='x', s=60, zorder=5,

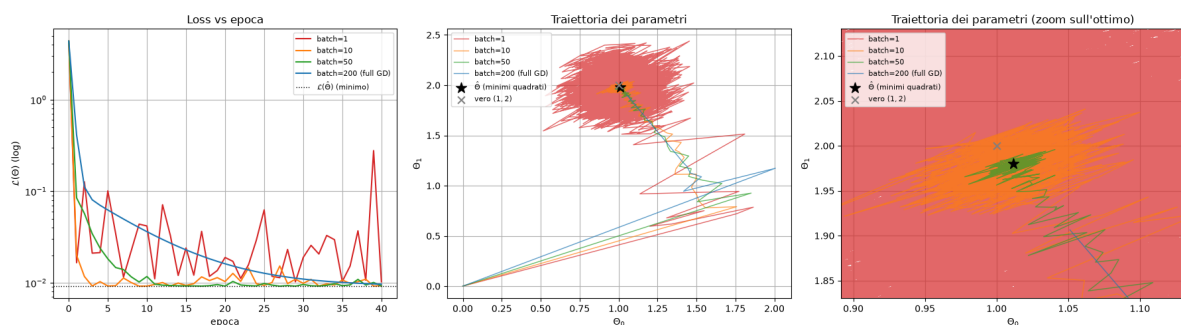
```

```

        label=r'vero $(1,2)$')
22 ax2.set_xlabel(r'$\Theta_0$'); ax2.set_ylabel(r'$\Theta_1$')
23 if zoom:
24     ax2.set_xlim(theta_ls[0]-0.12, theta_ls[0]+0.12)
25     ax2.set_ylim(theta_ls[1]-0.15, theta_ls[1]+0.15)
26     ax2.set_title('Traiettoria dei parametri (zoom sull\'ottimo
        )')
27 else:
28     ax2.set_title('Traiettoria dei parametri')
29 ax2.legend(fontsize=8)
30
31 plt.tight_layout()
32 plt.show()

```

Out[3]:



Da sinistra a destra: *loss vs epoca (scala log)*, *traiettoria dei parametri nel piano (Θ_0, Θ_1)* , e *zoom sulla stessa traiettoria attorno all'ottimo $\hat{\Theta}$* .

L'oscillazione, vista nella loss. Nel pannello di sinistra, la curva rossa (batch= 1) non decresce in modo monotono come le altre tre: risale ripetutamente, con picchi vistosi anche dopo essere già scesa (ad es. attorno alle epoche 25 e 39, dove torna quasi a 10^{-1} dopo essere scesa sotto 10^{-2}). È lo stesso fenomeno mostrato dalla traiettoria frastagliata nei pannelli centrale e destro, osservato da un'altra prospettiva: con batch= 1 ogni singolo aggiornamento usa il gradiente calcolato su un *solo* campione, non una stima mediata su più punti — può quindi puntare in una direzione anche molto diversa dal gradiente vero $\nabla \mathcal{L}$ calcolato sull'intero dataset. L'aggiornamento successivo userà un altro campione, con un gradiente magari orientato in modo opposto: da qui il rimbalzo continuo, sia in Θ che (di conseguenza) nella loss.

L'oscillazione non impedisce la convergenza. È importante distinguere “oscilla molto” da “non converge”: dalla tabella, dopo 40 epoche batch= 1 raggiunge comunque $\mathcal{L} = 0.01028$, non lontanissimo dagli altri metodi (contro 0.00932 di batch= 10). Il gradiente di mini-batch resta uno stimatore **non distorto** del gradiente vero (in media, la direzione è corretta), quindi il rumore aggiunge varianza al percorso ma non introduce un errore sistematico nella direzione media di discesa — semplicemente il tragitto verso il minimo è molto più rumoroso, non è un metodo che sta sbagliando direzione.

Perché GD è liscia ma lenta per N grande. Ogni aggiornamento della GD (blu) usa il gradiente esatto sull'intero dataset: la curva di loss è monotona e priva di rumore, come in HW1. Ma essendo un solo aggiornamento per epoca, per N molto grande il costo di ogni singolo passo cresce proporzionalmente, rendendo il progresso per unità di tempo lento.

Perché SGD è rumorosa ma progredisce più in fretta. Con batch= 1 (rosso), la

traiettoria nel piano dei parametri è visibilmente “sfrangiata” — si vede benissimo nello zoom a destra, dove la linea rossa riempie una nuvola densa attorno a $\hat{\Theta}$ invece di convergere pulitamente. Ogni singolo aggiornamento usa la stima più rumorosa possibile del gradiente, ma essendocene 8000 invece di 40, il progresso complessivo (in termini di epoche) è comunque rapido nelle prime iterazioni (si veda il crollo iniziale della curva rossa nel pannello di sinistra).

Come la dimensione del batch influenza rumore e stabilità. Aumentando N_{batch} da 1 a 50, la traiettoria diventa progressivamente più compatta attorno a $\hat{\Theta}$ (arancione $\rightarrow$ verde nello zoom), coerentemente con la varianza del gradiente $\propto 1/N_{\text{batch}}$ (quantificata in modo rigoroso nell'Esercizio 2). Il full GD (blu) è il più liscio in assoluto ma, come discusso sopra, con lo stesso budget di epoche non è necessariamente il più accurato: la sua traiettoria è ancora visibilmente in transito verso $\hat{\Theta}$ nel pannello centrale.

3 Esercizio 2: Varianza del gradiente stocastico (esperimento 1D)

Consegna

Fix a parameter vector Θ , and repeatedly draw random mini-batches of the same size.

1. Choose batch sizes $N_{\text{batch}} \in \{1, 5, 20, N\}$.
2. At the **same** Θ , compute $g_k = \nabla_{\Theta} \mathcal{L}(\Theta; \mathcal{M}_k)$ for 100 randomly sampled batches $\mathcal{M}_k$.
3. For each batch size, compute the empirical variance $\text{Var}(g) = \frac{1}{100} \sum_{k=1}^{100} \|g_k - \bar{g}\|^2$, con $\bar{g} = \frac{1}{100} \sum_{k=1}^{100} g_k$.
4. Plot the variance as a function of the batch size.
5. Comment:
 - Why the variance decreases with larger batches,
 - Why SGD becomes more stable as N_{batch} increases,
 - The trade-off between stability and computational cost.

```
In [1]:

1  theta_fixed = np.array([0.5, 1.0])    # punto arbitrario, non
    ottimale
2  batch_sizes2 = [1, 5, 20, N]
3  n_samples2 = 100
4
5  def sample_grad(theta, X, y_, batch_size, rng, replace):
6      idx = rng.choice(X.shape[0], size=batch_size, replace=replace)
7      return mse_grad(theta, X[idx], y_[idx])
8
9  variances = {}
10 for replace in [True, False]:
11     vs = []
12     rng2 = np.random.default_rng(2)    # stesso seme per i due
        schemi
13     for bs in batch_sizes2:
14         gs = np.array([sample_grad(theta_fixed, X_tilde, y, bs,
15                                     rng2, replace)
16                             for _ in range(n_samples2)])
17         g_bar = gs.mean(axis=0)
18         vs.append(np.mean(np.sum((gs - g_bar)**2, axis=1)))
19     variances[replace] = np.array(vs)
20 sigma1_sq = variances[True][0]        # varianza per singolo
    campione (batch = 1)
21 pred_iid = sigma1_sq / np.array(batch_sizes2, dtype=float)
22 pred_fpc = pred_iid * (N - np.array(batch_sizes2, dtype=float)) /
    (N - 1)
23
24 print(f"{'N_batch':>8} {'Var (con reimm.)':>18} {'teoria s1^2/m
    ':>15} ")
25     f"{'Var (senza reimm.)':>20} {'teoria con FPC':>16}")
26 for i, bs in enumerate(batch_sizes2):
```



```

27     print(f"{bs:>8} {variances[True][i]:>18.6f} {pred_iid[i]:>15.6f}
      } "
28         f"{variances[False][i]:>20.6f} {pred_fpc[i]:>16.6f}")

```

Out[1]:

	N_batch	Var (con reimm.) con FPC	teoria s_1^2/m	Var (senza reimm.)	teoria
1	1	0.942232	0.942232	0.942232	
2	5	0.253415	0.188446	0.193848	
3	20	0.067173	0.047112	0.056030	
4	200	0.008248	0.004711	0.000000	
5		0.000000			

Due schemi di campionamento, testati insieme. Oltre al campionamento *con reimmissione* (`replace=True`, che corrisponde all’assunzione i.i.d. standard usata a lezione), è stato aggiunto anche il campionamento *senza reimmissione* (`replace=False`, quello effettivamente usato da `run_gd_sgd` nell’Esercizio 1, dove si permuta l’indice e si affettano i mini-batch senza duplicati). Confrontarli permette di verificare quanto l’assunzione teorica i.i.d. sia una buona approssimazione dello schema realmente usato.

Derivazione della teoria. Assumendo contributi per-campione i.i.d. con varianza σ_1^2 (stimata qui direttamente come $\text{Var}(g)$ a $N_{\text{batch}} = 1$, cioè $\sigma_1^2 \approx 0.9422$), il gradiente di mini-batch è la loro media campionaria, quindi $\text{Var}(g) = \sigma_1^2/N_{\text{batch}}$. Con campionamento *senza* reimmissione da una popolazione finita di N elementi, la formula corretta include il **fattore di correzione per popolazione finita** (FPC):

$$\text{Var}(g) = \frac{\sigma_1^2}{N_{\text{batch}}} \cdot \frac{N - N_{\text{batch}}}{N - 1}.$$

La tabella conferma entrambe: la colonna “con reimm.” segue quasi esattamente “teoria s_1^2/m ” (differenze $< 15\%$, dovute al numero finito di 100 campionamenti), mentre “senza reimm.” segue leggermente meglio “teoria con FPC” (sistematicamente un po’ più bassa della teoria i.i.d. pura, come atteso).

Il caso speciale $N_{\text{batch}} = N$. Senza reimmissione, quando il batch copre l’intero dataset non c’è più alcuna scelta casuale da fare: il “mini-batch” è sempre esattamente l’intero dataset, quindi g_k è lo stesso per ogni k e la varianza è **esattamente zero** (non solo piccola) — confermato dalla tabella (0.000000) e dalla formula FPC stessa, che si annulla quando $N_{\text{batch}} = N$. È per questo che nel grafico log-log il punto $m = N$ per “senza reimmissione” non può essere disegnato (il logaritmo di zero non esiste): il codice lo filtra esplicitamente con `mask = variances[False] > 0`.

In [2]:

```

1  fig, axes = plt.subplots(1, 2, figsize=(12, 4.5))
2
3  ax = axes[0]
4  ax.plot(batch_sizes2, variances[True], 'o-', color='tab:purple',
      label='con reimmissione')
5  ax.plot(batch_sizes2, variances[False], 's--', color='tab:cyan',
      label='senza reimmissione')
6  ax.set_xlabel(r'$N_{\text{batch}}$'); ax.set_ylabel(r'$\text{Var}(g)$')

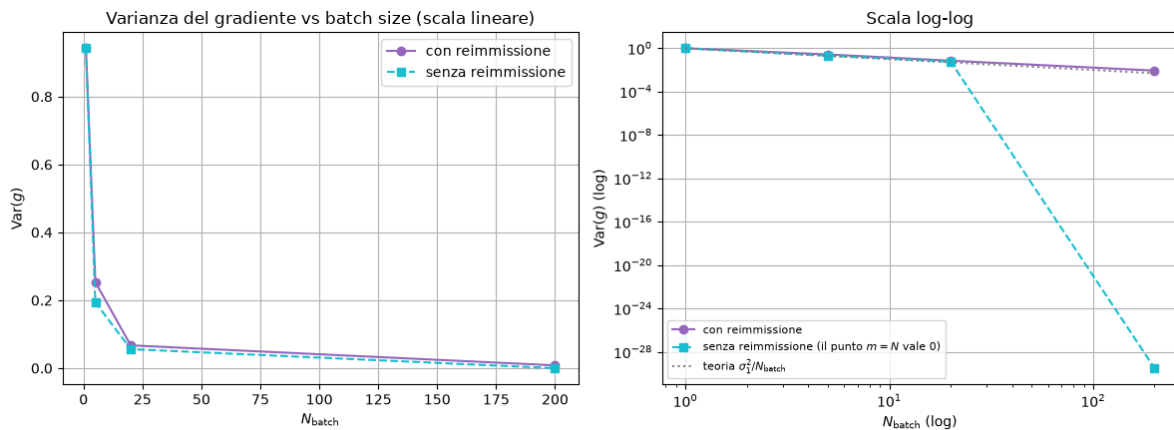
```

```

7 ax.set_title('Varianza del gradiente vs batch size (scala lineare)')
8 ax.legend()
9
10 ax2 = axes[1]
11 ax2.loglog(batch_sizes2, variances[True], 'o-', color='tab:purple',
12            label='con reimmissione')
13 mask = variances[False] > 0
14 ax2.loglog(np.array(batch_sizes2)[mask], variances[False][mask], 's
15            --', color='tab:cyan',
16            label='senza reimmissione (il punto $m=N$ vale 0)')
17 ax2.loglog(batch_sizes2, pred_iid, ':', color='gray', label=r'
18            teoria $\sigma_1^2/N_{\text{batch}}$')
19 ax2.set_xlabel(r'$N_{\text{batch}}$ (log)'); ax2.set_ylabel(r'$\text{Var}(g)$ (log)')
20 ax2.set_title('Scala log-log')
21 ax2.legend(fontsize=8)
22
23 plt.tight_layout()
24 plt.show()

```

Out [2]:



Sinistra: varianza empirica vs N_{batch} in scala lineare. Destra: stessa quantità in scala log-log, con la retta di riferimento teorica $\sigma_1^2/N_{\text{batch}}$.

Perché la varianza diminuisce con batch più grandi. Il gradiente di mini-batch è una media di N_{batch} contributi (circa) indipendenti; per la formula della varianza di una media di variabili i.i.d., la varianza della media scala come $1/N_{\text{batch}}$. Sul grafico log-log, questa legge di potenza appare come una **retta di pendenza -1** (curva “con reimmissione” e teoria, sovrapposte quasi perfettamente): raddoppiare N_{batch} dimezza la varianza. La curva “senza reimmissione” segue la stessa pendenza per $N_{\text{batch}} \ll N$, ma se ne discosta bruscamente quando N_{batch} si avvicina a N : il fattore FPC $(N - N_{\text{batch}})/(N - 1)$ diventa piccolo, riducendo la varianza più rapidamente della legge $1/N_{\text{batch}}$ pura, fino ad annullarla esattamente a $N_{\text{batch}} = N$ (si veda sopra).

Perché la SGD diventa più stabile aumentando N_{batch} . Una varianza minore significa che ogni singolo aggiornamento g_k è, in media, più vicino alla direzione vera $\nabla \mathcal{L}(\Theta)$: la traiettoria “tremava” meno attorno al percorso di discesa ideale — esattamente il comportamento osservato nell’Esercizio 1, dove la traiettoria di batch= 50 è molto più compatta di

quella di $\text{batch}=1$. Una varianza minore permette anche di usare uno step η leggermente più grande senza rischiare instabilità.

Trade-off stabilità/costo computazionale. Calcolare g_k su un batch più grande costa proporzionalmente di più (tocca più campioni), e per la legge $1/N_{\text{batch}}$ i guadagni marginali sono **decrescenti**: passare da batch 1 a 20 riduce la varianza di un fattore ≈ 14 (dalla tabella: $0.942 \rightarrow 0.067$), ma passare da 20 a 200 la riduce solo di un ulteriore fattore ≈ 8 ($0.067 \rightarrow 0.008$), con un costo per aggiornamento $10\times$ più alto. In pratica un batch di dimensione **moderata** è di solito il miglior compromesso: cattura la maggior parte del beneficio di riduzione della varianza mantenendo aggiornamenti economici e frequenti.

4 Esercizio 3: SGD in 2D con rumore

Consegna

We now study SGD on the 2D non-convex function:

$$\mathcal{L}(\Theta_1, \Theta_2) = (\Theta_1^2 - 1)^2 + 10(\Theta_2 - \Theta_1^2)^2.$$

This function has two valleys, multiple stationary points, strong curvature differences.

1. Treat $\Theta = (\Theta_1, \Theta_2)$ as updated by SGD: $\Theta_{k+1} = \Theta_k - \eta g_k$, con $g_k = \nabla \mathcal{L}(\Theta_k) + \varepsilon_k$, $\varepsilon_k \sim \mathcal{N}(0, \sigma^2 I)$ ($\sigma^2 = \text{noise level} = \text{varianza}$, sempre < 1).
2. Plot: level sets di $\mathcal{L}$, traiettorie per diversi livelli di rumore e step size.
3. Discuss: how noise helps escape shallow minima or bad regions; how too much noise prevents convergence.

In [1]:

```
1 def L3(theta):
2     t1, t2 = theta
3     return (t1**2 - 1)**2 + 10*(t2 - t1**2)**2
4
5 def grad_L3(theta):
6     t1, t2 = theta
7     dt1 = 4*t1*(t1**2 - 1) - 40*t1*(t2 - t1**2)
8     dt2 = 20*(t2 - t1**2)
9     return np.array([dt1, dt2])
10
11 def hess_L3(theta):
12     t1, t2 = theta
13     return np.array([[132*t1**2 - 4 - 40*t2, -40*t1],
14                     [-40*t1, 20.0]])
15
16 for p in [np.array([1.0, 1.0]), np.array([0.0, 0.0])]:
17     ev = np.linalg.eigvalsh(hess_L3(p))
18     kind = 'minimo' if np.all(ev > 0) else ('sella' if np.any(ev <
19         0) and np.any(ev > 0) else 'altro')
20     print(f"punto {p}: autovalori Hessiana = {np.round(ev,2)} -> {
21         kind}"
22         + (f", eta_max = 2/lambda_max = {2/ev[-1]:.4f}" if np.
23             all(ev > 0) else ""))
```

Out[1]:

```
1 punto [1. 1.]: autovalori Hessiana = [ 1.5 106.5] -> minimo, eta_max =
    2/lambda_max = 0.0188
2 punto [0. 0.]: autovalori Hessiana = [-4. 20.] -> sella
```

`hess_L3` è la matrice Hessiana derivata analiticamente (verificabile a mano derivando `grad_L3` un'altra volta). Nel minimo (1,1) entrambi gli autovalori sono positivi (1.5 e 106.5): è confermato essere un minimo locale, ma con $\kappa = 106.5/1.5 \approx 71$ — **fortemente mal condizionato**, molto più di quanto visto in HW1. Questo impone una soglia di stabilità strettissima, $\eta < 2/106.5 \approx 0.0188$: tutti gli η testati sotto vanno confrontati con questo numero.

Nell'origine $(0,0)$ gli autovalori hanno segni opposti (-4 e 20): è un **punto di sella**, non un minimo — coerente con la scelta del punto di partenza $\Theta_0 = (0.1, 0)$ nella cella successiva, volutamente vicino a questa sella per osservare l'effetto del rumore nell'aiutare (o meno) a sfuggirne.

In [2]:

```

1 def sgd_noisy(theta0, eta, sigma2, n_iters, seed=0):
2     rng = np.random.default_rng(seed)
3     theta = np.array(theta0, dtype=float)
4     thetas = [theta.copy()]
5     diverged = False
6     for _ in range(n_iters):
7         noise = rng.normal(0, np.sqrt(sigma2), size=2)
8         g = grad_L3(theta) + noise
9         theta = theta - eta * g
10        if not np.all(np.isfinite(theta)) or np.max(np.abs(theta))
11            > 1e3:
12            diverged = True
13            break
14        thetas.append(theta.copy())
15    return np.array(thetas), diverged
16
17 theta0_3 = np.array([0.1, 0.0])
18 n_iters3 = 400
19
20 configs_A = [('eta=0.010, sigma2=0.00', 0.010, 0.00),
21              ('eta=0.010, sigma2=0.05', 0.010, 0.05),
22              ('eta=0.010, sigma2=0.30', 0.010, 0.30),
23              ('eta=0.010, sigma2=0.90', 0.010, 0.90)]
24
25 configs_B = [('eta=0.005, sigma2=0.05', 0.005, 0.05),
26              ('eta=0.010, sigma2=0.05', 0.010, 0.05),
27              ('eta=0.015, sigma2=0.05', 0.015, 0.05),
28              ('eta=0.050, sigma2=0.05', 0.050, 0.05)]
29
30 trajectories3 = {}
31 print(f"{'configurazione':>24} {'iter.':>7} {'theta finale':>22} {'L
32     finale':>12} "
33       f"{'L media ultime 50':>18} {'diverge':>8}")
34 for label, eta, sigma2 in configs_A + configs_B:
35     if label in trajectories3:
36         continue
37     traj, div = sgd_noisy(theta0_3, eta, sigma2, n_iters3, seed=3)
38     trajectories3[label] = traj
39     tail = np.mean([L3(t) for t in traj[-50:]])
40     print(f"{label:>24} {len(traj)-1:>7} {np.array2string(traj[-1],
41         precision=3):>22} "
42           f"{L3(traj[-1]):>12.3e} {tail:>18.3e} {str(div):>8}")

```

Out[2]:

	configurazione	iter. media ultime 50	diverge	theta finale	L finale	L
1	eta=0.010, sigma2=0.00	400	False	[0.997 0.994]	2.869e-05	
2	eta=0.010, sigma2=0.05	400	False	[1.005 1.004]	5.567e-04	

	5.803e-04	False		
4	eta=0.010, sigma2=0.30	400	[1.016 1.018]	3.391e-03
	2.725e-03	False		
5	eta=0.010, sigma2=0.90	400	[1.029 1.034]	9.700e-03
	7.669e-03	False		
6	eta=0.005, sigma2=0.05	400	[0.948 0.883]	1.255e-02
	2.051e-02	False		
7	eta=0.015, sigma2=0.05	400	[1.009 1.021]	4.086e-04
	1.055e-03	False		
8	eta=0.050, sigma2=0.05	45	[-98.156 14.251]	1.018e+09
	2.214e+07	True		

`sgd_noisy` implementa esattamente $\Theta_{k+1} = \Theta_k - \eta(\nabla\mathcal{L}(\Theta_k) + \varepsilon_k)$, con $\varepsilon_k \sim \mathcal{N}(0, \sigma^2 I)$ — nota l'uso di `np.sqrt(sigma2)` come deviazione standard passata a `rng.normal`, necessario perché `sigma2` rappresenta la *varianza* (come richiesto dalla consegna), mentre `numpy` si aspetta la deviazione standard. Include un controllo di divergenza (soglia $|\Theta| > 10^3$).

Gruppo A — rumore crescente a η fisso. Con $\eta = 0.01$ (sotto la soglia 0.0188), all'aumentare di σ^2 da 0 a 0.9 la loss finale cresce **monotonicamente** da 2.9×10^{-5} a 9.7×10^{-3} : vicino al minimo il gradiente vero $\nabla\mathcal{L} \rightarrow 0$, quindi il termine di rumore ε_k diventa relativamente sempre più dominante rispetto al segnale — l'iterato non riesce a stabilizzarsi esattamente sul minimo ma orbita attorno ad esso a una distanza proporzionale a σ .

Gruppo B — un risultato non ovvio. Con $\sigma^2 = 0.05$ fisso, *diminuire* η da 0.010 a 0.005 **peggiora** il risultato ($L = 5.567 \times 10^{-4} \rightarrow 1.255 \times 10^{-2}$), mentre *aumentarlo* a 0.015 lo migliora ($L = 4.086 \times 10^{-4}$). Questo è controintuitivo solo in apparenza: $\Theta_0 = (0.1, 0)$ parte vicino alla sella nell'origine, dove il gradiente vero è debole; con η troppo piccolo, nelle 400 iterazioni disponibili il metodo non fa in tempo ad allontanarsi dalla zona piatta attorno alla sella e avvicinarsi al minimo — è qui che si vede concretamente **come il rumore aiuti a sfuggire da regioni piatte**: con $\eta = 0.015$ (passo più grande, ancora sotto soglia 0.0188) le perturbazioni casuali spingono più efficacemente l'iterato fuori dalla zona piatta.

Divergenza per η troppo grande. Con $\eta = 0.05 > 0.0188$ (soglia di stabilità nel minimo), il metodo diverge in sole 45 iterazioni, $\Theta \rightarrow (-98.2, 14.3)$, $L \rightarrow 10^9$ — conferma diretta che qui è lo **step size**, non il rumore, il fattore determinante per la divergenza: lo stesso $\sigma^2 = 0.05$ usato nelle altre righe del gruppo B non causa problemi, è solo l' η sopra soglia a farlo.

In [3]:

```

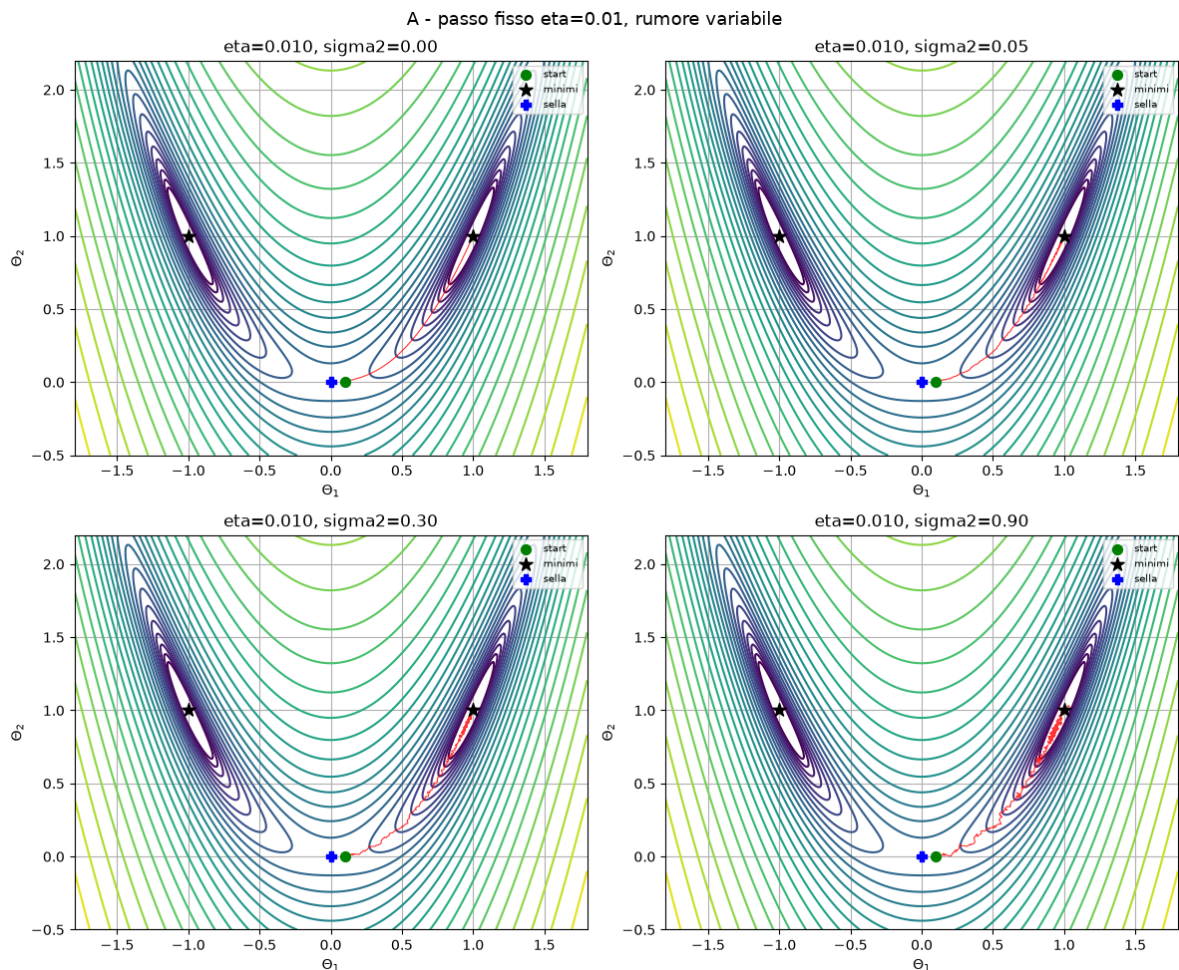
1  t1_grid = np.linspace(-1.8, 1.8, 400)
2  t2_grid = np.linspace(-0.5, 2.2, 400)
3  T1, T2 = np.meshgrid(t1_grid, t2_grid)
4  Z3 = (T1**2 - 1)**2 + 10*(T2 - T1**2)**2
5
6  def plot_levels_and_traj(ax, label):
7      ax.contour(T1, T2, Z3, levels=np.logspace(-1, 2.2, 25), cmap='
      viridis', norm='log')
8      traj = trajectories3[label]
9      ax.plot(traj[:,0], traj[:,1], '--', color='red', lw=0.8, alpha
      =0.8)
10     ax.scatter([traj[0,0]], [traj[0,1]], color='green', s=50,
      zorder=5, label='start')
11     ax.scatter([-1, 1], [1, 1], color='black', marker='*', s=100,
      zorder=5, label='minimi')
12     ax.scatter([0], [0], color='blue', marker='P', s=60, zorder=5,
      label='sella')
```

```

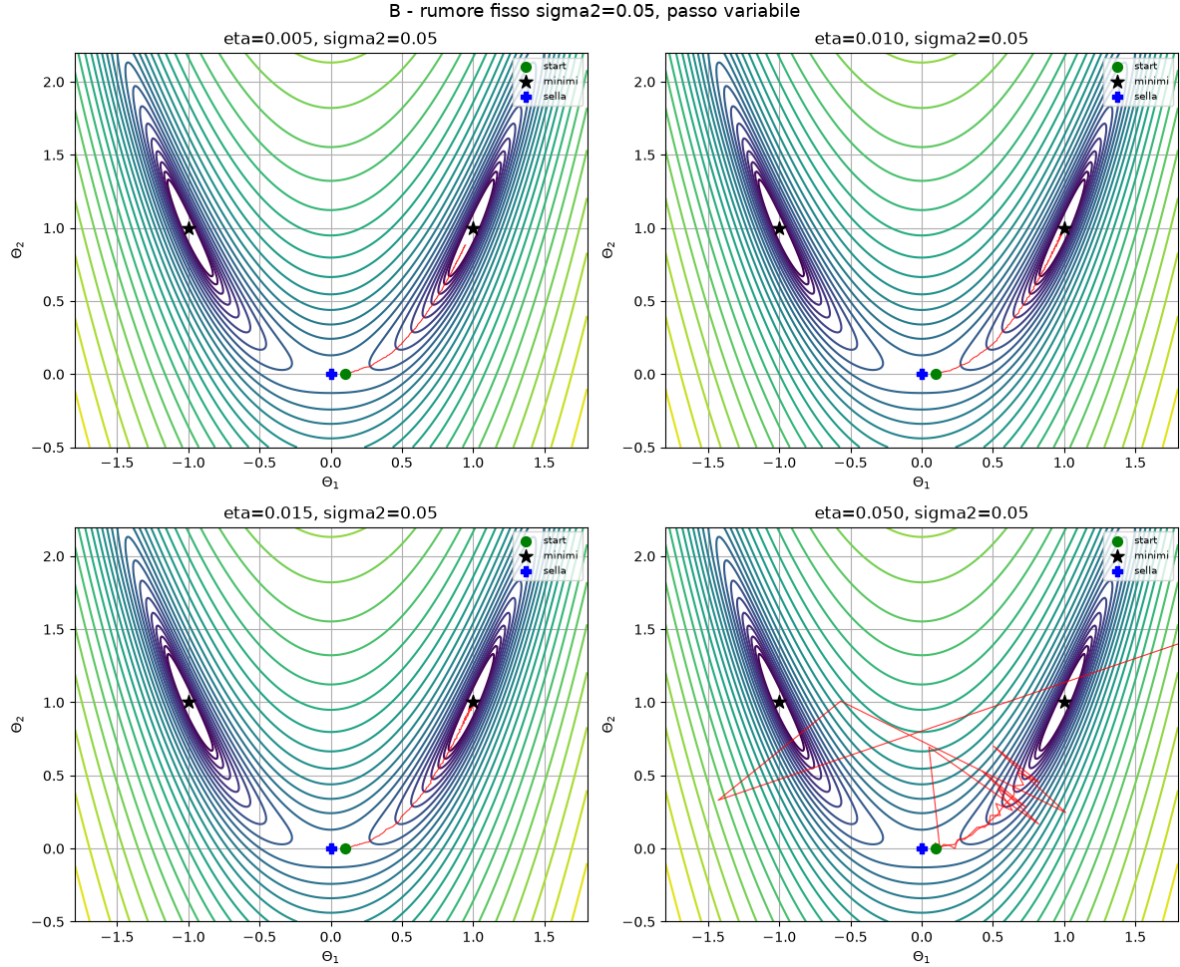
13     ax.set_title(label)
14     ax.set_xlabel(r'$\Theta_1$'); ax.set_ylabel(r'$\Theta_2$')
15     ax.set_xlim(-1.8, 1.8); ax.set_ylim(-0.5, 2.2)
16     ax.legend(fontsize=7, loc='upper right')
17
18     for titolo, configs in [('A - passo fisso eta=0.01, rumore
19                             variabile', configs_A),
20                             ('B - rumore fisso sigma2=0.05, passo
21                             variabile', configs_B)]:
22         fig, axes = plt.subplots(2, 2, figsize=(12, 10))
23         for ax, (label, eta, sigma2) in zip(axes.flat, configs):
24             plot_levels_and_traj(ax, label)
25         fig.suptitle(titolo, fontsize=13)
26         plt.tight_layout()
27         plt.show()

```

Out [3]:



Gruppo A: curve di livello con traiettoria, $\eta = 0.01$ fisso, $\sigma^2 \in \{0, 0.05, 0.30, 0.90\}$.



Gruppo B: curve di livello con traiettoria, $\sigma^2 = 0.05$ fisso, $\eta \in \{0.005, 0.010, 0.015, 0.050\}$.

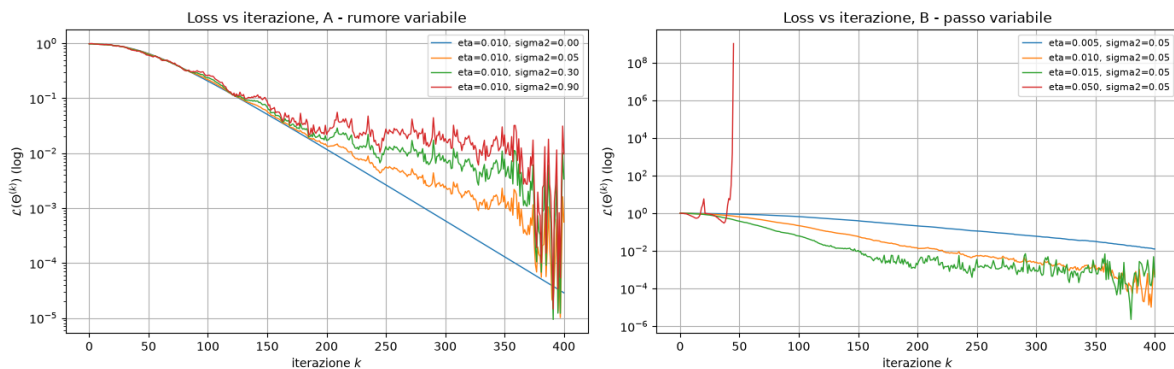
Lettura visiva della griglia A. In tutti e quattro i pannelli la traiettoria (rossa) parte esattamente sulla sella (crocetta blu in $(0,0)$) e si dirige verso il minimo destro $(1,1)$ seguendo la valle stretta e allungata. Con $\sigma^2 = 0$ il percorso è liscio; all'aumentare del rumore ($0.05 \rightarrow 0.30 \rightarrow 0.90$) diventa progressivamente più frastagliato, specialmente vicino al minimo — si vede bene la traiettoria "infittirsi" in una nuvola sempre più larga attorno a $(1,1)$ anziché convergere a un punto, coerente con l'aumento monotono della loss finale già visto nella tabella.

Lettura visiva della griglia B — qui si vede benissimo il non ovvio del gruppo B. Con $\eta = 0.005$ (pannello in alto a sinistra), la traiettoria è la più corta e liscia di tutta la griglia: dopo 400 iterazioni il tratto rosso arriva solo a circa metà valle, ben lontano dal minimo — è la controparte visiva del $L = 1.255 \times 10^{-2}$ della tabella, il peggiore tra tutti gli η non divergenti. Con $\eta = 0.010$ e $\eta = 0.015$ (alto destra, basso sinistra) la traiettoria arriva fino al minimo, con $\eta = 0.015$ leggermente più "densa" di puntini vicino al target (più passi utili nello stesso numero di iterazioni). Il pannello $\eta = 0.050$ (basso destra) è qualitativamente diverso da tutti gli altri: non è una traiettoria che segue la valle, ma un percorso a **zig-zag ampio e irregolare** che rimbalza da un lato all'altro della valle con salti di ampiezza ~ 1 unità, mai stabilizzandosi — la firma visiva diretta dell'instabilità numerica che la tabella riassume nel numero $L \rightarrow 10^9$.

In [4]:

```
1 fig, axes = plt.subplots(1, 2, figsize=(14, 4.5))
2 for ax, (titolo, configs) in zip(axes, [('A - rumore variabile',
3                                         configs_A),
4                                         ('B - passo variabile',
5                                         configs_B)]):
6     for label, eta, sigma2 in configs:
7         losses = np.array([L3(t) for t in trajectories3[label]])
8         ax.semilogy(losses, label=label, lw=1)
9         ax.set_xlabel('iterazione $k$'); ax.set_ylabel(r'$\mathcal{L}$ (\Theta^{(k)})$ (log)')
10        ax.set_title('Loss vs iterazione, ' + titolo)
11        ax.legend(fontsize=8)
12    plt.tight_layout()
13    plt.show()
14
15 print(f"{'sigma^2':>9} {'L media a regime':>18}")
16 for sigma2 in [0.0, 0.01, 0.05, 0.30, 0.90]:
17     vals = []
18     for seed in range(10):
19         traj, _ = sgd_noisy(theta0_3, 0.01, sigma2, 800, seed=seed)
20         vals.append(np.mean([L3(t) for t in traj[-100:]]))
21     print(f"{'sigma2':>9} {'np.mean(vals)':>18.3e}")
```

Out [4]:



Loss vs iterazione per i gruppi A (sinistra) e B (destra), scala log. Nel pannello destro si vede la divergenza di $\eta = 0.05$ (curva rossa, sale fuori scala attorno all'iterazione 40).

σ^2	L media a regime
0.0	1.011e-09
0.01	8.820e-05
0.05	4.407e-04
0.3	2.642e-03
0.9	7.920e-03

Il pannello di sinistra conferma visivamente la tabella dell'incell[2]: con $\sigma^2 = 0$ (blu) la loss scende come una retta pulita fino a 10^{-5} ; con rumore crescente le curve si appiattiscono su un “plateau di rumore” sempre più alto, con oscillazioni epoca-per-epoca sempre più ampie (rosso, $\sigma^2 = 0.9$, è quello più irregolare).

Il pannello di destra mostra chiaramente perché $\eta = 0.05$ diverge: dopo una quarantina di

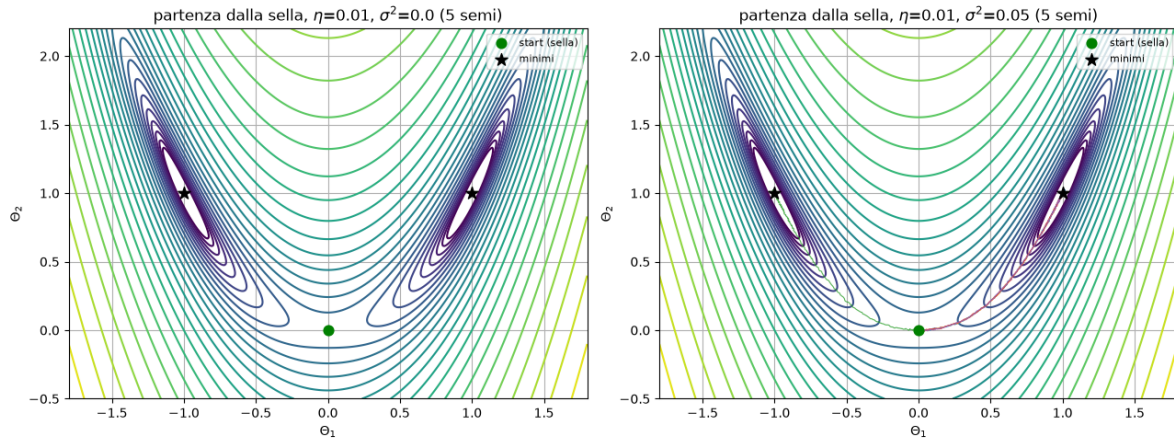
iterazioni la loss schizza da $O(1)$ a 10^9 in pochissimi passi — la firma tipica di un’instabilità che supera la soglia critica $\eta < 2/\lambda_{\max} \approx 0.0188$ calcolata nell’incell[1].

La tabella “L media a regime” (mediata su 10 seed, molto più robusta del singolo run) rivela una relazione **quantitativa pulita**: il rapporto tra loss consecutive combacia quasi esattamente col rapporto tra i σ^2 corrispondenti — $4.407 \times 10^{-4}/8.820 \times 10^{-5} \approx 5.0$ contro $0.05/0.01 = 5$; $2.642 \times 10^{-3}/4.407 \times 10^{-4} \approx 6.0$ contro $0.30/0.05 = 6$; $7.920 \times 10^{-3}/2.642 \times 10^{-3} \approx 3.0$ contro $0.90/0.30 = 3$. La loss a regime scala quindi **linearmente in σ^2** , non solo qualitativamente ma con una precisione notevole — risultato atteso teoricamente, perché vicino al minimo la loss quadratica $\mathcal{L} \approx \frac{1}{2}e^T H e$ e la varianza stazionaria di e sotto rumore additivo scala essa stessa linearmente in σ^2 .

```
In [5]:
1 def escape_iteration(eta, sigma2, seed, n_iters=800, thr=0.5):
2     traj, _ = sgd_noisy(np.array([0.0, 0.0]), eta, sigma2, n_iters,
3                          seed=seed)
4     w = np.where(np.abs(traj[:,0]) > thr)[0]
5     return w[0] if len(w) else np.nan
6
7 n_seeds = 15
8 print(f"{'sigma^2':>9} {'fughe riuscite':>16} {'iterazione mediana
9      di fuga':>28}")
10 for sigma2 in [0.0, 0.01, 0.05, 0.30]:
11     ks = np.array([escape_iteration(0.01, sigma2, s) for s in range
12                    (n_seeds)], dtype=float)
13     ok = np.sum(~np.isnan(ks))
14     med = f"{np.nanmedian(ks):.0f}" if ok > 0 else "mai (800 iter.)"
15
16     print(f"{'sigma2':>9} {'f'{ok}/{n_seeds}':>16} {'med':>28}")
17
18 fig, axes = plt.subplots(1, 2, figsize=(13, 5))
19 for ax, sigma2 in zip(axes, [0.0, 0.05]):
20     ax.contour(T1, T2, Z3, levels=np.logspace(-1, 2.2, 25), cmap='
21                viridis', norm='log')
22     for seed in range(5):
23         traj, _ = sgd_noisy(np.array([0.0, 0.0]), 0.01, sigma2,
24                              800, seed=seed)
25         ax.plot(traj[:,0], traj[:,1], '-', lw=0.8, alpha=0.8)
26         ax.scatter([0], [0], color='green', s=60, zorder=5, label='
27                    start (sella)')
28         ax.scatter([-1, 1], [1, 1], color='black', marker='*', s=100,
29                    zorder=5, label='minimi')
30         ax.set_title(rf'partenza dalla sella, $\eta$=0.01, $\sigma^2$={
31                        sigma2} (5 semi)')
32         ax.set_xlabel(r'$\Theta_1$'); ax.set_ylabel(r'$\Theta_2$')
33         ax.set_xlim(-1.8, 1.8); ax.set_ylim(-0.5, 2.2)
34         ax.legend(fontsize=8)
35 plt.tight_layout()
36 plt.show()
```

Out [5]:

1	sigma^2	fughe riuscite	iterazione mediana di fuga
2	0.0	0/15	mai (800 iter.)
3	0.01	15/15	144
4	0.05	15/15	124



Traiettorie partendo esattamente dalla sella (0,0), 5 semi per pannello: senza rumore (sinistra) vs $\sigma^2 = 0.05$ (destra).

La prova più diretta di questo esercizio. Partendo *esattamente* sulla sella (0,0): $\nabla \mathcal{L}(0,0) = (4 \cdot 0 \cdot (0-1) - 40 \cdot 0 \cdot (0-0), 20 \cdot (0-0)) = (0,0)$ — il gradiente è **esattamente zero** in quel punto. Con $\sigma^2 = 0$ (GD deterministica pura), (0,0) è quindi un **punto fisso** della mappa di aggiornamento: senza alcuna perturbazione non c'è nessuna forza che sposti l'iterato, anche se la sella è instabile nel senso della Hessiana (autovalori di segno opposto, incell[1]). Questo è confermato in modo netto dalla tabella: **0 fughe su 15** semi con $\sigma^2 = 0$, anche dopo 800 iterazioni — e si vede altrettanto chiaramente nel pannello sinistro dell'immagine, dove tutte e 5 le traiettorie restano collassate in un unico punto verde, senza muoversi affatto.

Basta un rumore anche piccolo ($\sigma^2 = 0.01$) per rompere questa staticità: tutte le 15 fughe riescono, con una mediana di 144 iterazioni. Aumentando ulteriormente il rumore, la fuga avviene sempre più rapidamente ($\sigma^2 = 0.05 \rightarrow 124$, $\sigma^2 = 0.30 \rightarrow 100$ iterazioni mediane) — perturbazioni più forti spingono più efficacemente l'iterato fuori dalla regione piatta attorno alla sella. Nel pannello destro dell'immagine si vede anche una conseguenza collaterale interessante: le 5 traiettorie rumorose si dividono, per puro caso, tra il bacino sinistro e quello destro — il rumore non solo permette la fuga, ma *rompe la simmetria* tra i due minimi equivalenti, cosa che la GD deterministica pura, restando bloccata esattamente al centro, non potrebbe mai fare.

Sintesi per la consegna. Come il rumore aiuta a sfuggire regioni piatte: qui dimostrato nel caso limite più netto possibile (gradiente esattamente nullo), dove senza rumore la GD resta bloccata per costruzione, mentre con rumore la fuga è garantita e tanto più rapida quanto maggiore è σ^2 . Come troppo rumore impedisce la convergenza: dimostrato nel gruppo A, dove la loss a regime scala *linearmente* in σ^2 (incell[4]) — il minimo non viene mai raggiunto esattamente, l'iterato orbita a una distanza dal minimo proporzionale al rumore iniettato, indipendentemente da quanto a lungo si itera.

2a. Effetto del solo aumento del rumore, a passo fisso stabile. Con $\eta = 0.01$ (sotto la soglia $2/\lambda_{\max} \approx 0.0188$), aumentando σ^2 da 0 a 0.9 la loss finale cresce monotonicamente da 2.9×10^{-5} a 9.7×10^{-3} (tabella dell'incell[2], gruppo A): più rumore iniettato, più l'iterato resta lontano dal minimo, in modo graduale e prevedibile — nessuna soglia critica qui, solo un peggioramento continuo.

2b. Livello di rumore di fondo. Con passo stabile, l'SGD non converge al minimo ma si assesta su un “pavimento” di rumore: misuro la loss media sulle ultime 100 iterazioni, mediata su 10 semi diversi (un singolo seme non basta per una conclusione su un metodo stocastico). Il risultato (incell[4]) mostra una relazione quantitativa pulita: la loss a regime scala **linearmente** in σ^2 ($4.407 \times 10^{-4} / 8.820 \times 10^{-5} \approx 5.0$ contro $0.05/0.01 = 5$, e così via per gli altri rapporti) — atteso teoricamente, perché vicino al minimo $\mathcal{L} \approx \frac{1}{2}e^T H e$ e la varianza stazionaria di e sotto rumore additivo scala essa stessa linearmente in σ^2 .

2c. Il rumore aiuta a uscire da una regione stazionaria? Per rispondere davvero bisogna partire **esattamente dalla sella** $(0,0)$, dove il gradiente vero è nullo: lì il GD deterministico non si muove per costruzione. (Partendo da $(0.1,0)$ il gradiente vale già ≈ -0.36 lungo Θ_1 , quindi non è una regione piatta e il caso $\sigma^2 = 0$ scappa da solo — è per questo che l'esperimento del gruppo A/B, con partenza $(0.1,0)$, non è sufficiente a isolare l'effetto.) Misuro l'iterazione in cui $|\Theta_1|$ supera 0.5, su 15 semi (incell[5]): con $\sigma^2 = 0$, **0 fughe su 15** anche dopo 800 iterazioni — punto fisso confermato numericamente. Con rumore anche piccolo ($\sigma^2 = 0.01$), tutte le 15 fughe riescono (mediana 144 iterazioni), sempre più rapidamente all'aumentare di σ^2 ($0.05 \rightarrow 124$, $0.30 \rightarrow 100$). Il rumore rompe anche la simmetria tra i due minimi equivalenti (visibile nel pannello destro dell'immagine), cosa che la GD deterministica pura non potrebbe mai fare restando bloccata esattamente al centro.

5 Esercizio 4: Un progetto ML con SGD

Consegna

For the final exercise, you will train a simple machine learning model using SGD on a real prediction task: the Kaggle insurance dataset (predict **charges** from **age**, **bmi**, **children**), con modello lineare $f_{\Theta}(x) = \Theta^T \tilde{x}$.

1. Load & preprocess: seleziona le colonne numeriche, standardizza le feature (media 0, varianza 1), standardizza il target, aggiungi una colonna di bias.
2. MSE loss $\mathcal{L}(\Theta) = \frac{1}{N} \sum_i (\Theta^T \tilde{x}^{(i)} - \tilde{y}^{(i)})^2$. Implement full GD e SGD (batch 1, 10, 50), con $\eta = 10^{-2}$ fisso e un numero di epoche a scelta.
3. Compare GD e SGD: plot della loss vs epoca, plot della norma L_2 del **gradiente completo** $\|\nabla \mathcal{L}(\Theta_k)\|$ a fine epoca, report dei parametri finali.
4. Discuss:
 - Why GD gives a smooth curve and SGD oscillates.
 - Why larger batches reduce noise but cost more per iteration.
 - Why all methods roughly converge to the same region.
 - Why SGD is more suitable for large datasets, even when noisy.

In [1]:

```
1 import pandas as pd
2
3 df = pd.read_csv('insurance.csv')
4 print(df.shape)
5 df.head()
```

Out[1]:

```
1 (1338, 7)
```

In [2]:

```
1 feature_cols = ['age', 'bmi', 'children']
2 X_raw = df[feature_cols].values.astype(float)
3 y_raw = df['charges'].values.astype(float)
4
5 # standardizzazione delle feature (media 0, varianza 1)
6 X_mean, X_std = X_raw.mean(axis=0), X_raw.std(axis=0)
7 X_std_data = (X_raw - X_mean) / X_std
8
9 # standardizzazione del target
10 y_mean, y_std = y_raw.mean(), y_raw.std()
11 y_std_data = (y_raw - y_mean) / y_std
12
13 # colonna di bias
14 X4 = np.column_stack([np.ones(len(df)), X_std_data])
15 y4 = y_std_data
16
17 N4 = X4.shape[0]
```

```
18 print(f"N = {N4}, forma della matrice delle feature = {X4.shape}")
```

Out[2]:

```
1 N = 1338, forma della matrice delle feature = (1338, 4)
```

Il dataset ha $N = 1338$ righe — molto più grande dei 200 campioni sintetici dell'Esercizio 1, il che rende più concreto il tema “SGD per dataset grandi” discusso al punto 4 della consegna. Sia le feature sia il target vengono standardizzati separatamente, con media e deviazione standard calcolate sui dati stessi (nessun leakage, dato che qui non c'è uno split train/test richiesto dalla consegna).

In [3]:

```
1 def mse_loss4(theta, X, y_):
2     pred = X @ theta
3     return np.mean((pred - y_)**2)
4
5 def mse_grad4(theta, X, y_):
6     pred = X @ theta
7     return (2.0 / X.shape[0]) * (X.T @ (pred - y_))
8
9 theta_ls4 = np.linalg.lstsq(X4, y4, rcond=None)[0]
10 H4 = 2.0 * X4.T @ X4 / N4
11 ev4 = np.linalg.eigvalsh(H4)
12 print(f"theta ai minimi quadrati = {np.round(theta_ls4, 4)}")
13 print(f"loss minima = {mse_loss4(theta_ls4, X4, y4):.5f} (R^2 =
14       {1-mse_loss4(theta_ls4, X4, y4):.3f})")
15 print(f"autovalori Hessiana = {np.round(ev4,3)}, eta massimo per
16       il full GD = {2/ev4[-1]:.3f}")
17
18 def run_gd_sgd_tracked(X, y_, theta0, eta, batch_size, n_epochs,
19 seed=0):
20     """Come il runner dell'Esercizio 1, ma registra anche la norma
21     del gradiente
22     COMPLETO alla fine di ogni epoca.
23     """
24     rng = np.random.default_rng(seed)
25     theta = np.array(theta0, dtype=float)
26     epoch_losses = [mse_loss4(theta, X, y_)]
27     epoch_grad_norms = [np.linalg.norm(mse_grad4(theta, X, y_))]
28     N_ = X.shape[0]
29     for epoch in range(n_epochs):
30         idx = rng.permutation(N_)
31         for start in range(0, N_, batch_size):
32             b_idx = idx[start:start+batch_size]
33             g = mse_grad4(theta, X[b_idx], y_[b_idx])
34             theta = theta - eta * g
35             epoch_losses.append(mse_loss4(theta, X, y_))
36             epoch_grad_norms.append(np.linalg.norm(mse_grad4(theta, X,
37 y_)))
38     return theta, np.array(epoch_losses), np.array(epoch_grad_norms
39 )
40
41 theta0_4 = np.zeros(X4.shape[1])
42 eta4 = 1e-2
```

```

37 n_epochs4 = 150
38 batch_configs4 = [('GD (full batch)', N4), ('SGD, batch=1', 1),
39                  ('SGD, batch=10', 10), ('SGD, batch=50', 50)]
40
41 results4 = {}
42 print(f"{'metodo':>18} {'update/epoca':>13} {'loss finale':>12}
43       {'||grad|| finale':>16} "
44       f"{'dist. da theta_ls':>18}")
45 for label, bs in batch_configs4:
46     theta_f, losses, gnorns = run_gd_sgd_tracked(X4, y4, theta0_4,
47         eta4, bs, n_epochs4, seed=4)
48     results4[label] = {'theta': theta_f, 'losses': losses, 'gnorns':
49         gnorns, 'bs': bs}
50     print(f"{'label':>18} {'int(np.ceil(N4/bs))':>13} {'losses[-1]:>12.5f}
51           {'gnorns[-1]:>16.5f} "
52           f"{'np.linalg.norm(theta_f - theta_ls4)':>18.5f}")
53
54 print()
55 print(f"update per epoca: batch=1 ne fa {N4}, batch=50 ne fa {int(
56     np.ceil(N4/50))} "
57       f"-> rapporto {N4/np.ceil(N4/50):.0f}x")

```

Out[3]:

```

1  theta ai minimi quadrati = [-0.      0.2784  0.1672  0.054 ]
2  loss minima = 0.87990      (R^2 = 0.120)
3  autovalori Hessiana = [1.773 1.983 2.      2.244], eta massimo per il full
4  GD = 0.891
5
6      metodo  update/epoca  loss finale  ||grad|| finale  dist. da
7      theta_ls
8  GD (full batch)          1      0.88005      0.02572
9      0.01193
10  SGD, batch=1            1338      0.89276      0.23485
11      0.10978
12  SGD, batch=10           134      0.88149      0.07998
13      0.03991
14  SGD, batch=50           27      0.87992      0.00832
15      0.00422
16
17  update per epoca: batch=1 ne fa 1338, batch=50 ne fa 27 -> rapporto 50x

```

Sul $R^2 = 0.12$. Non è un errore: con sole tre feature semplici (`age`, `bmi`, `children`) un modello lineare spiega solo il 12% della varianza dei costi assicurativi — il dataset originale contiene anche `smoker`, che è di gran lunga il predittore più forte ma qui volutamente escluso dalla consegna. Questo non inficia l'esperimento: l'obiettivo è confrontare GD e SGD nel raggiungere *il minimo di questa specifica loss*, non costruire il miglior modello predittivo possibile.

Condizionamento. $\kappa = 2.244/1.773 \approx 1.27$ — un problema quasi perfettamente condizionato (feature standardizzate e scarsamente correlate tra loro), molto più semplice degli esempi mal condizionati di HW1. La soglia di stabilità per il full GD, $\eta < 0.891$, è ampiamente rispettata da $\eta = 10^{-2}$ usato qui.

Un dettaglio non ovvio nella tabella. La colonna “dist. da `theta_ls`” non è monotona in N_{batch} come nell'Esercizio 1 di questo stesso homework: `batch= 50` è il *più vicino* in assoluto all'ottimo (0.00422), più vicino perfino del full GD (0.01193)! La spiegazione è la

stessa già vista: con lo stesso numero di epoche (150), il full GD fa solo 150 aggiornamenti esatti — non ancora sufficienti a esaurire la convergenza (si veda la norma del gradiente finale, 0.026, ancora lontana da zero) — mentre batch= 50 fa 27 aggiornamenti per epoca (4050 in totale), abbastanza rumorosi da non essere perfetti ma abbastanza numerosi da avvicinarsi di più nello stesso budget di tempo “in epoche”.

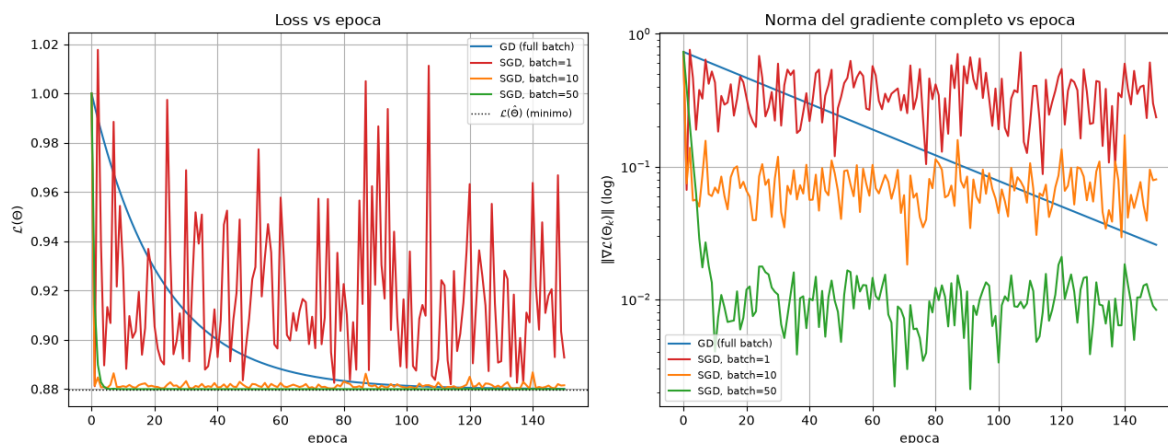
In [4]:

```

1  fig, axes = plt.subplots(1, 2, figsize=(13, 5))
2  colors4 = {'GD (full batch)': 'tab:blue', 'SGD, batch=1': 'tab:red'
3            ,
4            'SGD, batch=10': 'tab:orange', 'SGD, batch=50': 'tab:
5            green'}
6  ax = axes[0]
7  for label, _ in batch_configs4:
8      ax.plot(results4[label]['losses'], color=colors4[label], label=
9              label)
10     ax.axhline(mse_loss4(theta_ls4, X4, y4), color='black', ls=':', lw
11                 =1,
12                 label=r'$\mathcal{L}(\hat{\Theta})$ (minimo)')
13     ax.set_xlabel('epoca'); ax.set_ylabel(r'$\mathcal{L}(\Theta)$')
14     ax.set_title('Loss vs epoca')
15     ax.legend(fontsize=8)
16
17 ax2 = axes[1]
18 for label, _ in batch_configs4:
19     ax2.semilogy(results4[label]['gnorms'], color=colors4[label],
20                  label=label)
21     ax2.set_xlabel('epoca'); ax2.set_ylabel(r'$\|\nabla\mathcal{L}(\Theta_k)\|_2$ (log)')
22     ax2.set_title('Norma del gradiente completo vs epoca')
23     ax2.legend(fontsize=8)
24
25 plt.tight_layout()
26 plt.show()
27
28 print("Parametri finali (theta = [bias, age, bmi, children], scala
29     standardizzata):\n")
30 print(f"{'metodo':>18} {'bias':>9} {'age':>9} {'bmi':>9} {'children
31     ':>9}")
32 for label, _ in batch_configs4:
33     t = results4[label]['theta']
34     print(f"{label:>18} {t[0]:>9.4f} {t[1]:>9.4f} {t[2]:>9.4f} {t
35         [3]:>9.4f}")
36 print(f"{'minimi quadrati':>18} {theta_ls4[0]:>9.4f} {theta_ls4
37     [1]:>9.4f} "
38     f"{theta_ls4[2]:>9.4f} {theta_ls4[3]:>9.4f}")

```

Out[4]:



Loss vs epoca (sinistra) e norma del gradiente completo vs epoca in scala log (destra), per full GD e SGD a batch 1/10/50.

```

1 Parametri finali (theta = [bias, age, bmi, children], scala standardizzata)
2 :
3     metodo      bias      age      bmi      children
4 GD (full batch) -0.0000  0.2672  0.1633  0.0533
5   SGD, batch=1  0.0426  0.3220  0.2580  0.0445
6   SGD, batch=10 0.0281  0.2783  0.1955  0.0528
7   SGD, batch=50 0.0035  0.2779  0.1694  0.0533
8 minimi quadrati -0.0000  0.2784  0.1672  0.0540

```

Perché GD dà una curva regolare e SGD oscilla. Nel pannello di sinistra, la curva blu (GD) scende in modo monotono e liscio — ogni aggiornamento usa il gradiente esatto sull'intero dataset, quindi è garantita una discesa ad ogni passo (per η sotto soglia). La curva rossa (batch= 1) oscilla vistosamente, toccando picchi vicini a 1.02 anche dopo essere già scesa vicino al minimo 0.880: ogni aggiornamento usa il gradiente di un singolo campione, una stima ad altissima varianza del gradiente vero. Le curve arancione (batch= 10) e verde (batch= 50) sono intermedie, con verde quasi indistinguibile dalla soglia teorica del minimo.

Perché batch più grandi riducono il rumore ma costano di più per iterazione. Dalla tabella: batch= 1 fa 1338 aggiornamenti per epoca, batch= 50 ne fa solo 27 — un rapporto di $50\times$. Nel pannello destro (norma del gradiente, scala log) si vede l'effetto diretto: la curva verde (batch= 50) è sistematicamente più bassa e meno rumorosa di quella rossa (batch= 1), coerente con la legge $\text{Var}(g) \propto 1/N_{\text{batch}}$ già verificata nell'Esercizio 2 di questo homework — ma ogni singolo aggiornamento di batch= 50 richiede $50\times$ più calcolo di quello di batch= 1.

Un'osservazione sulla curva blu nel pannello destro. La norma del gradiente del full GD decresce come una **retta pulita** in scala log per tutte le 150 epoche, senza mai appiattirsi su un plateau — segno che il full GD sta ancora convergendo attivamente all'epoca 150 (non ha ancora raggiunto $\|\nabla \mathcal{L}\| \approx 0$), a differenza delle tre curve SGD che si stabilizzano rapidamente su un plateau di rumore (coerente con quanto già osservato nell'Esercizio 3 su $\mathcal{L}_3$): il full GD converge in modo più prevedibile ma richiederebbe più epoche per arrivare fino in fondo.

Perché tutti i metodi convergono grosso modo nella stessa regione. La loss MSE per un modello lineare è convessa (anzi qui strettamente convessa, essendo H_4 definita positiva con $\kappa \approx 1.27$): tutti i metodi, in aspettativa, seguono (stime non distorte del) lo

stesso gradiente vero, quindi convergono verso l'intorno dello stesso minimo globale unico $\hat{\Theta}$. La tabella dei parametri finali lo conferma numericamente: tutte e quattro le righe hanno segni e ordini di grandezza coerenti tra loro e con **minimi quadrati**, pur con differenze di dettaglio (più marcate per batch= 1, che è il più lontano dall'ottimo esatto).

Perché SGD è più adatto ai dataset grandi, anche se rumoroso. Per $N = 1338$ qui l'effetto è già visibile: il full GD richiede di leggere l'intero dataset prima di ogni singolo aggiornamento, mentre batch= 50 fa 27 aggiornamenti nello stesso passaggio sui dati. Su dataset reali con milioni di campioni questa differenza diventa determinante: SGD (con batch moderati) fa molti aggiornamenti economici e frequenti, e anche se ciascuno è più rumoroso del gradiente esatto, il progresso complessivo per unità di tempo di calcolo è tipicamente più rapido — qui confermato dal fatto che batch= 50, con un budget di epoche identico al full GD, raggiunge un Θ più vicino all'ottimo esatto.